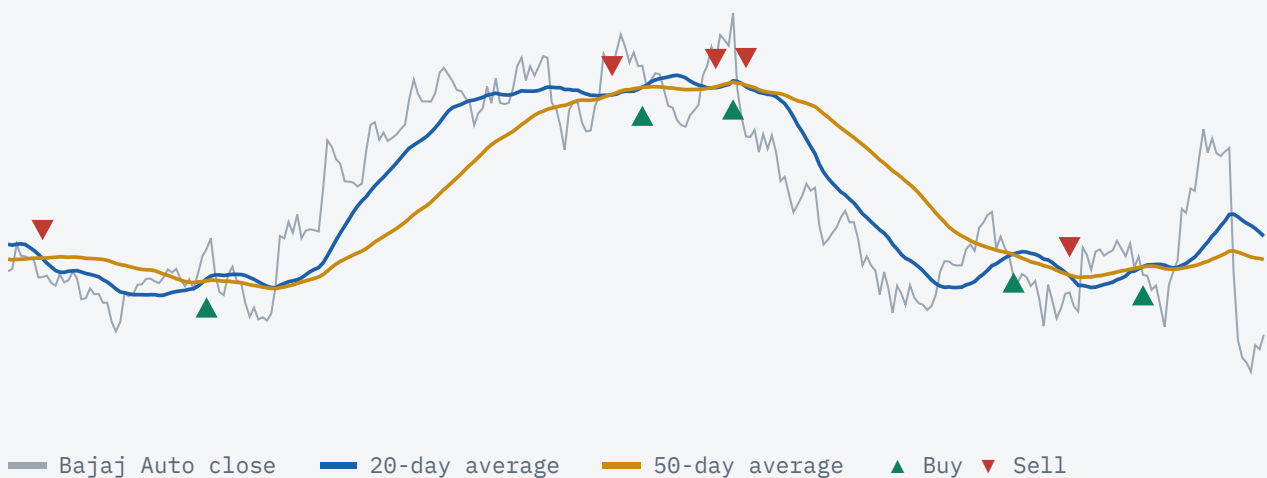


Stock market analysis in SQL

A step-by-step guide to moving averages, golden-cross signals, and the data problem hiding in plain sight. Six NSE stocks, January 2015 to July 2018.



Practise in the playground: <https://claude.ai/artifact/HLynNwi13TpkmQVRHFbCMx>

Before you start

This guide doesn't give you the answers. For each task it gives you the idea you need, the steps in order, a code skeleton with blanks to fill, and **checkpoints** — specific numbers your result must match. If your numbers match the checkpoints, you're right. If they don't, the common-mistakes box usually says why.

What you'll submit

1. Your analysis as a **.sql file**. 2. A **PDF of your insights**. 3. A short **video** walking through your work.

Where to write your SQL

Use the [online playground](#) to practise. It runs a real database in your browser, needs no installation, and every task has a **Check answer** button that tells you whether your result is right without showing you the solution. When you're confident, move your queries to MySQL for your submission. Appendix A shows how to load the CSVs there.

How each task is laid out

A **concept box** explains the one new idea, with a toy example.

Check yourself lists numbers your result must match.

Common mistakes are the bugs most students hit.

Still stuck? gives nudges. Read one, try again, then read the next.

The data

Six tables — `bajaj_auto`, `eicher_motors`, `hero_motocorp`, `infosys`, `tcs`, `tvsmotors` — each with 889 rows, one per trading day, and the same columns:

Column	Meaning
<code>date</code>	Trading day, stored as YYYY-MM-DD text
<code>open_price</code> , <code>high_price</code> , <code>low_price</code> , <code>close_price</code>	Prices in rupees. This project uses close_price .
<code>wap</code>	Weighted average price for the day
<code>no_of_shares</code> , <code>no_of_trades</code>	Volume traded and number of trades
<code>total_turnover</code>	Value traded in rupees
<code>deliverable_qty</code> , <code>pct_delivered</code>	Shares actually delivered, and as % of shares traded
<code>spread_high_low</code> , <code>spread_close_open</code>	High – low, and close – open

Playground (SQLite) vs MySQL

You want	Playground (SQLite)	MySQL 8
Year from a date	<code>strftime('%Y', date)</code>	<code>YEAR(date)</code>
A column called <code>signal</code>	<code>signal</code>	<code>`signal`</code> — reserved word, needs backticks
Reusable function	not available; write the query	<code>CREATE FUNCTION</code> (Appendix B)
Window functions	supported	supported from MySQL 8.0 only

1

Part 1: get to know the data

How much history do we have?

Goal. Return the number of trading days, the first date and the last date in `bajaj_auto`, in that column order.

You'll use: `COUNT`, `MIN`, `MAX`, column aliases

- 1 Run `SELECT * FROM bajaj_auto LIMIT 5;` and look at the columns. Notice the date format: `YYYY-MM-DD`.
- 2 Count the rows. Every row is one trading day.
- 3 Find the earliest and latest date. Because the dates are stored as `YYYY-MM-DD` text, sorting them alphabetically is the same as sorting them by time, so `MIN` and `MAX` work.
- 4 Give each column a readable name with `AS`.

```
SELECT COUNT(*) AS trading_days,  
       MIN(date) AS first_day,  
       MAX(date) AS last_day  
FROM bajaj_auto;
```

Check yourself

- One row, three columns.
- Just under 900 trading days.
- The first day is New Year's Day 2015; the last is the final day of July 2018.

Common mistakes

- Dates stored as `'31-July-2018'` would *not* sort correctly — `'31'` comes after `'1'` but so does `'4'`. That's why the playground stores ISO dates, and why the MySQL loader in the appendix converts them.

2

Part 1: get to know the data

Eicher's five best closes

Goal. Return the `date` and `close_price` of Eicher Motors' five highest closing prices, highest first.

You'll use: `ORDER BY ... DESC`, `LIMIT`

- 1 Select the two columns from `eicher_motors`.
- 2 Sort so the biggest close is on top.
- 3 Keep only the first five rows.

```
SELECT date, close_price
FROM eicher_motors
ORDER BY ____ ____
LIMIT ____;
```

Check yourself

- Five rows.
- The top close is above ₹32,000.
- All five dates fall in the same month. Which month? That's a nice line for your insights PDF.

Common mistakes

- Forgetting `DESC` gives you the five *lowest* closes.
- Putting `LIMIT` before `ORDER BY` is a syntax error — the order of clauses is fixed.

3

Part 1: get to know the data

TCS, year by year

Goal. Return each `year` and TCS's average closing price for that year, rounded to 2 decimals, oldest year first.

You'll use: `strftime`, `GROUP BY`, `AVG`, `ROUND`

Grouping by something you calculate

You don't have a `year` column, but you can make one. In SQLite, `strftime('%Y', date)` returns '2015' for '2015-03-12'. In MySQL the same thing is `YEAR(date)`.

`GROUP BY` collapses all rows that share a value into one row, and aggregates like `AVG` summarise each group.

- 1 Write a `SELECT` that shows `date` next to `strftime('%Y', date)`. Run it. Confirm the year column looks right.
- 2 Now replace `date` with `AVG(close_price)` and add a `GROUP BY` on the year.
- 3 Round the average to 2 decimals.
- 4 Sort by year.

```
SELECT strftime('%Y', date) AS year,  
       ____ (____(close_price), 2) AS avg_close  
FROM tcs  
GROUP BY ____  
ORDER BY ____;
```

Check yourself

- Four rows: 2015, 2016, 2017, 2018.
- The 2016 average is exactly 2419.00.
- 2018 only has seven months of data. Keep that in mind before you call 2018 the 'best year'.

Common mistakes

- `GROUP BY date` gives you 889 groups of one row each — no summarising at all.

4

Part 1: get to know the data

Find the holes

Goal. Across all six tables, return the `stock` (use the table name) and the `date` of every row where `deliverable_qty` is NULL.

You'll use: `IS NULL`, string literals as columns, `UNION ALL`

Stacking results with UNION ALL

`UNION ALL` puts the rows of one SELECT underneath another. Both SELECTs must return the same number of columns in the same order. A text literal like `'tcs' AS stock` adds a column that says which table each row came from — you'll reuse this trick in tasks 10–13.

`UNION` without `ALL` also removes duplicate rows, which costs time and can silently delete rows you wanted. Use `ALL` unless you mean to de-duplicate.

- 1 Write the query for one table: `SELECT 'bajaj_auto' AS stock, date FROM bajaj_auto WHERE ...`
- 2 Run it. You should get exactly one row.
- 3 Copy it five times, change the table name and the literal each time, and join them with `UNION ALL`.

Check yourself

- Six rows in total, one per stock.
- Those six rows fall on only two distinct dates. What does that suggest about where the gap came from — the companies, or the exchange's reporting?

Common mistakes

- `WHERE deliverable_qty = NULL` never matches anything. `NULL` is not a value, it's the absence of one, so nothing is 'equal' to it. Use `IS NULL`.
- Putting a semicolon after each SELECT ends the statement early; only the last one gets a semicolon.

Moving averages

Goal. Create a table `bajaj1` with `date`, `close_price`, `ma20`, `ma50`. `ma20` is the average close of the current day and the previous 19 trading days; `ma50` the same over 50 days. Round both to 2 decimals. Leave them NULL until a full window of days exists.

You'll use: window functions: `AVG()` `OVER`, `ROWS BETWEEN`, `ROW_NUMBER()`, `CASE`, `CREATE TABLE ... AS`

How a window function works

A normal aggregate like `AVG(close_price)` squashes many rows into one. A **window function** computes the aggregate *for every row*, looking at a 'window' of neighbouring rows, and keeps all rows.

Here is a toy table with a **3-day** moving average. The window for each row is 'this row and the 2 before it':

day	price	rows in the window	3-day avg
1	10	10	<i>should be NULL</i>
2	12	10, 12	<i>should be NULL</i>
3	11	10, 12, 11	11
4	13	12, 11, 13	12
5	15	11, 13, 15	13

In SQL, that window is written `AVG(price) OVER (ORDER BY day ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`. On days 1 and 2 SQL still returns a number — it just averages whatever rows exist. A '3-day' average of one day is not a 3-day average, so you have to blank those rows out yourself.

`ROW_NUMBER() OVER (ORDER BY day)` tells you which row you're on.

Your job is to adapt this from 3 days to 20 and 50.

- 1 Start small: `SELECT date, close_price FROM bajaj_auto ORDER BY date;`
- 2 Add a raw 20-day average using the pattern above. How many `PRECEDING` rows make a 20-row window? (Not 20.)
- 3 Run it and look at the first row. Its 'average' equals its own close price. That's the partial-window problem.
- 4 Add `ROW_NUMBER() OVER (ORDER BY date) AS rn` as a temporary column so you can see the row numbers.
- 5 Wrap the average in a `CASE` that only returns it when the row number is big enough. Delete the temporary `rn` column.
- 6 Do the same for 50 days, then add `ROUND(..., 2)` around each average.
- 7 When the `SELECT` is right, put `DROP TABLE IF EXISTS bajaj1;` and `CREATE TABLE bajaj1 AS` in front of it.
- 8 Switch the playground to **Chart** view on `SELECT * FROM bajaj1;`. You should see two smooth lines weaving around the price.

```

DROP TABLE IF EXISTS bajaj1;
CREATE TABLE bajaj1 AS
SELECT
    date,
    close_price,
    CASE WHEN ROW_NUMBER() OVER (ORDER BY date) >= 20
         THEN ROUND(AVG(close_price) OVER (ORDER BY date ROWS BETWEEN 20 PRECEDING AND CURRENT ROW), 2)
    END AS ma20,
    -- ma50: same idea, you write it
FROM bajaj_auto;

```

Check yourself

- 889 rows, 4 columns, in the order date, close_price, ma20, ma50.
- The first non-NULL ma20 is on **2015-01-29** and equals **2415.53**.
- The first non-NULL ma50 is on **2015-03-13** and equals **2283.80**.
- On 2018-07-31, ma20 is **2918.51**.
- If your first ma20 lands on 2015-01-30 instead, you've built a 21-day window.

Common mistakes

- **Off by one.** ROWS BETWEEN 20 PRECEDING AND CURRENT ROW is 21 rows.
- **Missing ORDER BY inside OVER.** Without it the 'previous rows' are in no defined order.
- **Partial windows.** Skipping the CASE means the first 19 and 49 rows carry fake averages, and those fake values can trigger fake signals in task 7.
- Putting a comma after the last column before FROM.

Still stuck? Read one nudge at a time

1. The frame is ROWS BETWEEN n PRECEDING AND CURRENT ROW. For a 20-row window, $n + 1 = 20$.
2. The CASE condition compares the row number with the window size: row 20 is the first row that has 20 rows behind it (including itself).
3. Build and run the ma20 column on its own first. Only then copy it for ma50.

Master table

Goal. Create `master_table` with one row per date and these columns in order: `date`, `bajaj`, `tcs`, `tv`s, `infosys`, `eicher`, `hero`, each holding that stock's closing price.

You'll use: `JOIN ... ON`, table aliases

- 1 Before joining anything, check the assumption: do all six tables trade on the same days? Run a `COUNT(*)` of `bajaj_auto` joined to `tcs` on date. If that's 889, the dates line up.
- 2 Start from `bajaj_auto` `b` and join `tcs` `t` on the date. Select `b.date`, `b.close_price` AS `bajaj`, `t.close_price` AS `tcs`. Run it.
- 3 Add the other four joins one at a time, running after each.
- 4 Wrap it in `CREATE TABLE master_table AS`.

```
DROP TABLE IF EXISTS master_table;
CREATE TABLE master_table AS
SELECT b.date,
       b.close_price AS bajaj,
       t.close_price AS tcs,
       --- -- tvs, infosys, eicher, hero
FROM bajaj_auto b
JOIN tcs t ON t.date = b.date
JOIN --- ON ---
-- three more joins
;
```

Check yourself

- 889 rows, 7 columns, no NULLs anywhere.
- On 2018-07-31 the row reads `bajaj` 2700.70 and `tvs` 517.45.
- Chart it with the **Rebase to 100** box ticked. Which lines end highest? Keep that picture in mind for task 11.

Common mistakes

- Selecting `date` without an alias when several tables have a `date` column gives an 'ambiguous column' error.
- Columns in the wrong order. The checker (and anyone reading your table) expects the order given in the goal.
- If one table had missing days, an inner JOIN would silently drop those dates everywhere. Here it doesn't, but step 1 is how you'd find out.

Golden cross signals

Goal. Using `bajaj1`, create `bajaj2` with `date`, `close_price`, `signal`. Buy on the day `ma20` crosses above `ma50`, Sell on the day it crosses below, Hold on every other day — including days where today's or yesterday's averages are NULL.

You'll use: `LAG()`, a CTE (`WITH`), `CASE` with several conditions

A cross is about two days, not one

'`ma20` is above `ma50`' is true on hundreds of days in a row. A *crossover* happens on the single day that changes: yesterday it wasn't above, today it is.

date	ma20	ma50	yesterday ma20	yesterday ma50	signal
Mon	98	100	—	—	Hold
Tue	99	100	98	100	Hold
Wed	101	100	99	100	Buy
Thu	103	101	101	100	Hold
Fri	100	101	103	101	Sell

Wednesday is a Buy because yesterday `ma20 ≤ ma50` and today `ma20 > ma50`. Thursday is still above, but nothing changed, so it's Hold.

`LAG(ma20) OVER (ORDER BY date)` gives you the previous row's `ma20` — the 'yesterday' columns above. You can't use a window function inside a `WHERE` or refer to its alias in the same `SELECT`, so compute the lags in a CTE first, then write the `CASE` in the query that reads from it.

- 1 Write a CTE `t` that selects everything from `bajaj1` plus `prev_ma20` and `prev_ma50` using `LAG`.
- 2 Below it, `SELECT * FROM t LIMIT 60;` and eyeball rows 49–52. You should see `prev_ma50` become non-NULL one row after `ma50` does.
- 3 Replace that `SELECT` with `date`, `close_price` and a `CASE`. Put the NULL guard first: if either average you need is missing, it's Hold.
- 4 Add the Buy condition from the table above, then the Sell condition (the mirror image).
- 5 Everything else is Hold.
- 6 Wrap it in `CREATE TABLE bajaj2 AS`. Chart `bajaj1` joined to `bajaj2` to see the triangles land on the crossings.

```

DROP TABLE IF EXISTS bajaj2;
CREATE TABLE bajaj2 AS
WITH t AS (
    SELECT date, close_price, ma20, ma50,
           LAG(ma20) OVER (ORDER BY date) AS prev_ma20,
           LAG(ma50) OVER (ORDER BY date) AS prev_ma50
    FROM bajaj1
)
SELECT date, close_price,
CASE
    WHEN ma20 IS NULL OR prev_ma20 IS NULL THEN 'Hold'
    WHEN ma20 > ma50 AND prev_ma20 < prev_ma50 THEN 'Buy'
    WHEN ma20 < ma50 AND prev_ma20 > prev_ma50 THEN 'Sell'
    ELSE 'Hold'
END AS signal
FROM t;

```

Check yourself

- 889 rows, 3 columns.
- The very first Buy is on **2015-05-18**. The first Sell is on **2015-08-24**.
- Only a few dozen days in total should be Buy or Sell. If you have hundreds, read the first mistake below.

Common mistakes

- **Checking only today.** WHEN `ma20 > ma50` THEN 'Buy' marks 502 Bajaj days as Buy — every day of every uptrend.
- **Strict comparison on yesterday.** Using `prev_ma20 < prev_ma50` instead of `<=` misses the day after the two lines touched exactly.
- **CASE order.** CASE stops at the first true branch. If the NULL guard isn't first, a comparison with NULL falls through to ELSE — which happens to give Hold here, but the logic is accidental.
- Capitalisation: the checker expects exactly Buy, Sell, Hold.

Still stuck? Read one nudge at a time

1. `prev_ma50` is written exactly like `prev_ma20`, with the column name changed.
2. Today's `ma50` becomes non-NULL a day before yesterday's does. Which of the two do you need to guard?
3. Sell is the Buy line with both comparison signs flipped.

8

Part 2: the assignment

How often did it trigger?

Goal. From `bajaj2`, return each `signal` and how many days carry it, sorted by signal name.

You'll use: `GROUP BY`, `COUNT`

This task is short on purpose: it's your test for task 7.

- 1 Group `bajaj2` by signal and count.
- 2 Compare your Buy and Sell numbers with the counts quoted for Bajaj Auto in the course deck.

Check yourself

- Three rows: Buy, Hold, Sell.
- The three counts add up to 889.
- Buy and Sell differ by at most one. Think about why: after a Buy the next crossing must be a Sell.
- If your counts don't match the deck, the bug is in task 7 (or task 5), not here.

9

Part 2: the assignment

Signal on a given day

Goal. Return the signal from `bajaj2` for **2018-06-21**. In MySQL you'll turn this into a function (see the appendix).

You'll use: `WHERE` on a date; in MySQL, `CREATE FUNCTION`

- 1 Filter `bajaj2` to one date and return only the signal column.
- 2 Test it with a date where you already know the answer from task 7 before trusting it on 2018-06-21.
- 3 In MySQL, wrap it as a function using the skeleton in Appendix B.

Check yourself

- One row, one column.
- For 2015-05-18 your query returns Buy; for 2016-01-04 it returns Hold.
- A date the market was closed (a Sunday) returns no rows at all. In your MySQL function that means NULL — decide whether that's acceptable.

10

Part 2: the assignment

All six stocks in one query

Goal. Without creating twelve tables, return one row per `stock` with `buys`, `sells`, `last_signal_date`, `last_signal`, sorted by stock. Use the names Bajaj Auto, Eicher Motors, Hero Motocorp, Infosys, TCS, TVS Motors. Use unrounded averages.

You'll use: `PARTITION BY`, chained CTEs, `ROW_NUMBER()` for 'latest per group'

PARTITION BY: a separate window per group

Once all six stocks are stacked in one table, `ORDER BY date` alone would let Bajaj's 20-day average run into TCS's prices. `OVER (PARTITION BY stock ORDER BY date ...)` restarts the window for each stock, as if each had its own table.

'Latest row per group' is a classic pattern: number each group's rows newest-first with `ROW_NUMBER()` `OVER (PARTITION BY stock ORDER BY date DESC)`, then keep row 1.

- 1 **prices:** stack the six tables with a stock-name column (task 4's trick). Check: $6 \times 889 = 5,334$ rows.
- 2 **ma:** your task 5 averages, with `PARTITION BY stock` in every `OVER` — including the `ROW_NUMBER`. Leave out `ROUND`.
- 3 **lagged:** your task 7 lags, again partitioned.
- 4 **sig:** your task 7 CASE.
- 5 **latest:** non-Hold rows only, numbered newest-first per stock.
- 6 Final SELECT: count Buys and Sells per stock from `sig`, and join `latest` where the row number is 1.
- 7 Build it one CTE at a time. After each, end with `SELECT * FROM that_cte LIMIT 20;` and look before moving on.

```
WITH prices AS (
  SELECT 'Bajaj Auto' AS stock, date, close_price FROM bajaj_auto
  UNION ALL
  --- -- five more
),
ma AS (
  -- task 5 logic, PARTITION BY stock in every OVER
),
lagged AS (
  -- task 7 lags, partitioned
),
sig AS (
  -- task 7 CASE
),
latest AS (
  -- non-Hold rows, ROW_NUMBER() newest-first per stock
)
SELECT ---
FROM sig s
JOIN latest l ON ---
GROUP BY s.stock
ORDER BY s.stock;
```

Check yourself

- Six rows, five columns.
- The Bajaj Auto row matches your task 8 counts and your task 9 date.
- Across all six stocks there are **56** Buys and **57** Sells in total.
- Your numbers should match the course deck for every stock.

Common mistakes

- Partitioning the averages but not the LAG : each stock's first row 'remembers' the previous stock's last value.
- Forgetting to partition `ROW_NUMBER()` in the ma step, so only Bajaj gets NULLs for its first rows.
- Tip: a boolean like `signal = 'Buy'` is 1 or 0 in SQLite and MySQL, so `SUM(signal = 'Buy')` counts Buys.

11

Part 3: question the result

Who went up?

Goal. For each stock return `stock`, `first_close`, `last_close`, `pct_change` (1 decimal), comparing the first and last trading day, biggest gain first.

You'll use: CTEs, joining a table to itself, arithmetic

- 1 Reuse your **prices** CTE from task 10.
- 2 Add a CTE **ends** with each stock's `MIN(date)` and `MAX(date)`.
- 3 Join `ends` back to `prices` twice, once on the first date and once on the last, with different aliases.
- 4 % change = $(\text{last} - \text{first}) / \text{first} \times 100$. Multiply by `100.0`, not `100`.

Check yourself

- Six rows.
- TVS Motors tops the list at 86.9%.
- Two stocks come out negative. Compare them with the course deck's numbers: does every figure there match yours?
- Before moving on, write down which stocks you would call winners and losers. You'll revisit this.

Common mistakes

- Integer division: in SQLite, `1 / 2` is 0. `100.0 * ...` forces decimal maths.
- Using `MIN(close_price)` instead of the close on the *first date*. The lowest price and the first price are different things.

12

Part 3: question the result

The data trap

Goal. For each stock, find its single worst day: return `stock, date, close_price, pct_move` (day-over-day % change of close, 1 decimal), worst first.

You'll use: `LAG` with `PARTITION BY`, ranking with `ROW_NUMBER()`

- 1 From **prices**, compute each day's move: today's close divided by yesterday's, minus 1, times 100.
- 2 Remove the first day of each stock (its move is NULL).
- 3 Rank each stock's days from worst to best and keep rank 1.
- 4 Now look hard at your result. Then investigate the two rows that don't make sense: search the stock name and that date together, with the words 'bonus issue' or 'split'.

Check yourself

- Six rows, one per stock.
- Most stocks' worst day is somewhere between -6% and -10%.
- Two rows are wildly worse than the rest. Would a large, profitable company really lose that much value in one day with no headlines?

Common mistakes

- Not filtering out the NULL first day — NULLs sort to the top in SQLite and you'll get the first day of each stock as its 'worst'.

Think before task 13

If a price series has a jump that isn't a real change in value, what else in this project is affected? Consider your returns from task 11, the moving averages around that date, and the Buy/Sell signals near it.

Goal. Adjust the TCS and Infosys prices for the events you found in task 12, then return `stock` and `adjusted_pct_change` from 2015-01-01 to 2018-07-31 (1 decimal), sorted by stock.

You'll use: `CASE` inside a CTE, conditional aggregation

Adjusting a price series

In a 1:1 bonus issue, every shareholder gets one extra share for each share they own. The number of shares doubles, so the price halves overnight — but nobody gained or lost anything. To compare prices across that date, divide every price *before* the event by the adjustment factor (here, 2). Prices on and after the event date stay as they are.

The event date is the first day the price trades at the new level — the date you found in task 12.

- 1 Write a CTE **adjusted** with TCS rows, where `adj_close` is a CASE: before the event date, half the close; otherwise the close.
- 2 Add the Infosys rows with `UNION ALL`, using Infosys's own event date.
- 3 Pick out the first-day and last-day values per stock with `MAX(CASE WHEN date = '...' THEN adj_close END)`, and compute the % change.

```
WITH adjusted AS (
  SELECT 'TCS' AS stock, date,
         CASE WHEN date < '2015-01-01' THEN close_price / 2 ELSE close_price END AS adj_close
  FROM tcs
  UNION ALL
  SELECT 'INFOSYS' AS stock, date, close_price AS adj_close
  FROM infosys
)
SELECT stock,
       ROUND(100.0 * (MAX(adj_close) / MIN(adj_close) - 1), 1) AS adjusted_pct_change
FROM adjusted
GROUP BY stock
ORDER BY stock;
```

Check yourself

- Two rows.
- Chart `adj_close` for one stock. The cliff should be gone and the line continuous.
- Compare with task 11. Does the list of winners and losers you wrote down still hold?

Stretch challenge

Rebuild the TCS moving averages and signals on the adjusted prices. How many of the original TCS signals disappear, and which ones were created purely by the price cliff?

Writing your insights PDF

Your SQL shows what you computed. Your insights PDF shows what it means — and whether you trust it. For every claim you make, give three things: the **claim**, the **evidence** (which query, which number), and the **caveat** (what could make it wrong).

Questions to answer for each stock

- How did the price change over the full period — using adjusted prices where needed?
- How many Buy and Sell signals did it produce, and what was the most recent one?
- Were there stretches where signals flipped back and forth within a few weeks? What would following them have cost?
- Does the latest signal agree with the long-run trend? If not, which would you trust, and why?

Questions about the method

- A moving average only uses past prices. What does that mean for how *early* a golden cross can tell you anything?
- Which of the conclusions in the course deck change after task 13? Say which ones and by how much.
- What isn't in this data that a real investor would need (dividends, news, brokerage costs, the broader market)?

Before you submit

- ☐ Every task's result matches its checkpoints, in the playground and in MySQL.
- ☐ Your .sql file runs top to bottom on a fresh database without errors.
- ☐ Every table you create starts with `DROP TABLE IF EXISTS`, so the file can be re-run.
- ☐ Your insights PDF names the two price events from task 12 and says how you handled them.
- ☐ No number in your PDF is copied from the course deck without being checked against your own query.

Appendix A: loading the CSVs into MySQL

This is setup, not part of the assignment, so it's given in full. It converts the CSV dates (31-July-2018) into real dates and turns empty cells into NULL. Repeat for each file, changing the table and file names. Enable `local_infile` first: run `SET GLOBAL local_infile = 1;` and in Workbench add `OPT_LOCAL_INFILE=1` under the connection's Advanced tab.

```

CREATE TABLE bajaj_auto (
  `date` DATE PRIMARY KEY,
  open_price DECIMAL(12,2), high_price DECIMAL(12,2), low_price DECIMAL(12,2),
  close_price DECIMAL(12,2), wap DECIMAL(16,4),
  no_of_shares BIGINT, no_of_trades BIGINT, total_turnover DECIMAL(20,2),
  deliverable_qty BIGINT, pct_deli_qty DECIMAL(6,2),
  spread_high_low DECIMAL(12,2), spread_close_open DECIMAL(12,2)
);
LOAD DATA LOCAL INFILE '/path/to/Bajaj_Auto.csv' INTO TABLE bajaj_auto
FIELDS TERMINATED BY ',' LINES TERMINATED BY '\n' IGNORE 1 LINES
(@d,@o,@h,@l,@c,@w,@s,@tr,@to,@dq,@pd,@shl,@sco)
SET `date` = STR_TO_DATE(@d, '%d-%M-%Y'),
    open_price = NULLIF(@o,''), high_price = NULLIF(@h,''), low_price = NULLIF(@l,''),
    close_price = NULLIF(@c,''), wap = NULLIF(@w,''), no_of_shares = NULLIF(@s,''),
    no_of_trades = NULLIF(@tr,''), total_turnover = NULLIF(@to,''),
    deliverable_qty = NULLIF(@dq,''), pct_deli_qty = NULLIF(@pd,''),
    spread_high_low = NULLIF(@shl,''), spread_close_open = NULLIF(TRIM(@sco),'');

SELECT COUNT(*), MIN(`date`), MAX(`date`) FROM bajaj_auto;  -- check: 889 rows

```

Appendix B: a MySQL function skeleton

MySQL needs a temporary delimiter so the semicolons inside the function body don't end the `CREATE FUNCTION` statement early.

```

DELIMITER $$
CREATE FUNCTION bajaj_signal(d DATE)
RETURNS VARCHAR(4) DETERMINISTIC READS SQL DATA
BEGIN
  DECLARE s VARCHAR(4);
  SELECT ___ INTO s FROM ___ WHERE ___;
  RETURN s;
END $$
DELIMITER ;

SELECT bajaj_signal('2015-05-18');  -- check: Buy

```